

Step-by-step guide for installing Terraform on both Windows and Amazon Linux

=====

Terraform is an **open-source Infrastructure as Code (IaC) tool** created by HashiCorp.

It allows you to **define, provision, and manage infrastructure** (servers, networks, databases, cloud services, etc.) using simple configuration files written in **HCL (HashiCorp Configuration Language)**.

➤ **Key Points about Terraform**

- **Infrastructure as Code (IaC):** You write .tf files to describe infrastructure instead of manually creating it in the cloud.
- **Cloud-agnostic:** Works with multiple providers (AWS, Azure, GCP, VMware, Kubernetes, etc.).
- **Declarative:** You declare *what* you want, and Terraform figures out *how* to create it.
- **Idempotent:** Running the same code multiple times will not create duplicates—it ensures infrastructure matches your code.
- **Execution Plan:** Terraform shows what changes it will make (terraform plan) before actually applying them (terraform apply).

➤ **Terraform Workflow**

1. **Write** → Define infrastructure in .tf files.

2. **Init** → Initialize providers & plugins (terraform init).
3. **Plan** → Preview what Terraform will do (terraform plan).
4. **Apply** → Deploy the infrastructure (terraform apply).
5. **Destroy** → Tear down infrastructure when no longer needed (terraform destroy).

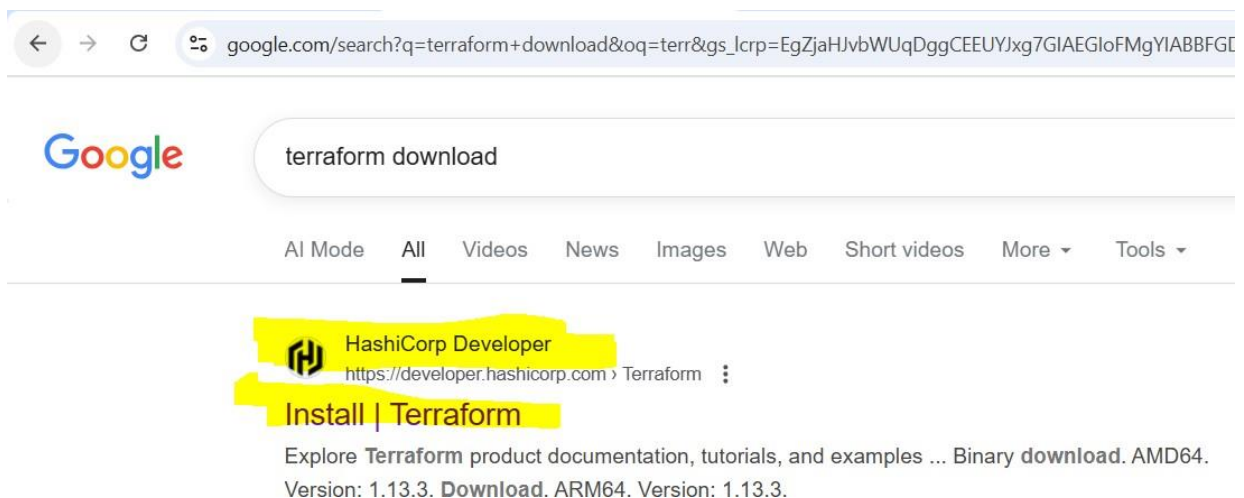
➤ **Difference Between Terraform vs Ansible: -**

Feature	Terraform	Ansible
Type	Infrastructure Provisioning Tool (IaC)	Configuration Management & Automation Tool
Main Purpose	To create, change, and destroy infrastructure (servers, networks, storage, cloud services, etc.)	To configure, deploy applications, and manage servers
Approach	Declarative – You define what you want (desired state), Terraform figures out how to get there.	Procedural (mostly) – You define step by step tasks to reach the state.
Execution Model	Builds an execution plan (terraform plan → terraform apply)	Executes tasks sequentially in playbooks

Feature	Terraform	Ansible
State Management	Maintains a state file to track resources	Stateless – does not track past states
Best For	Provisioning cloud infrastructure (AWS, Azure, GCP, VMware, etc.)	Configuring software, OS, and apps on provisioned servers
Idempotency	Ensures resources always match the desired code (no duplicates)	Can be idempotent, but depends on playbook design

Now install Terraform on Windows

Go to any browser type terraform download and choose first site and select windows 386



The screenshot shows the Terraform installation page. The left sidebar contains links to Terraform Home, Install Terraform, and Operating Systems (macOS, Windows, Linux, FreeBSD, OpenBSD, Solaris). The main content area shows the macOS installation instructions, including a package manager command and binary download links for AMD64 and ARM64. Below this, the Windows section is highlighted with a yellow background, showing binary download links for 386 and AMD64.

macOS

Package manager

```
brew tap hashicorp/tap  
brew install hashicorp/tap/terraform
```

Binary download

Architecture	Version	Download
AMD64	Version: 1.13.3	Download
ARM64	Version: 1.13.3	Download

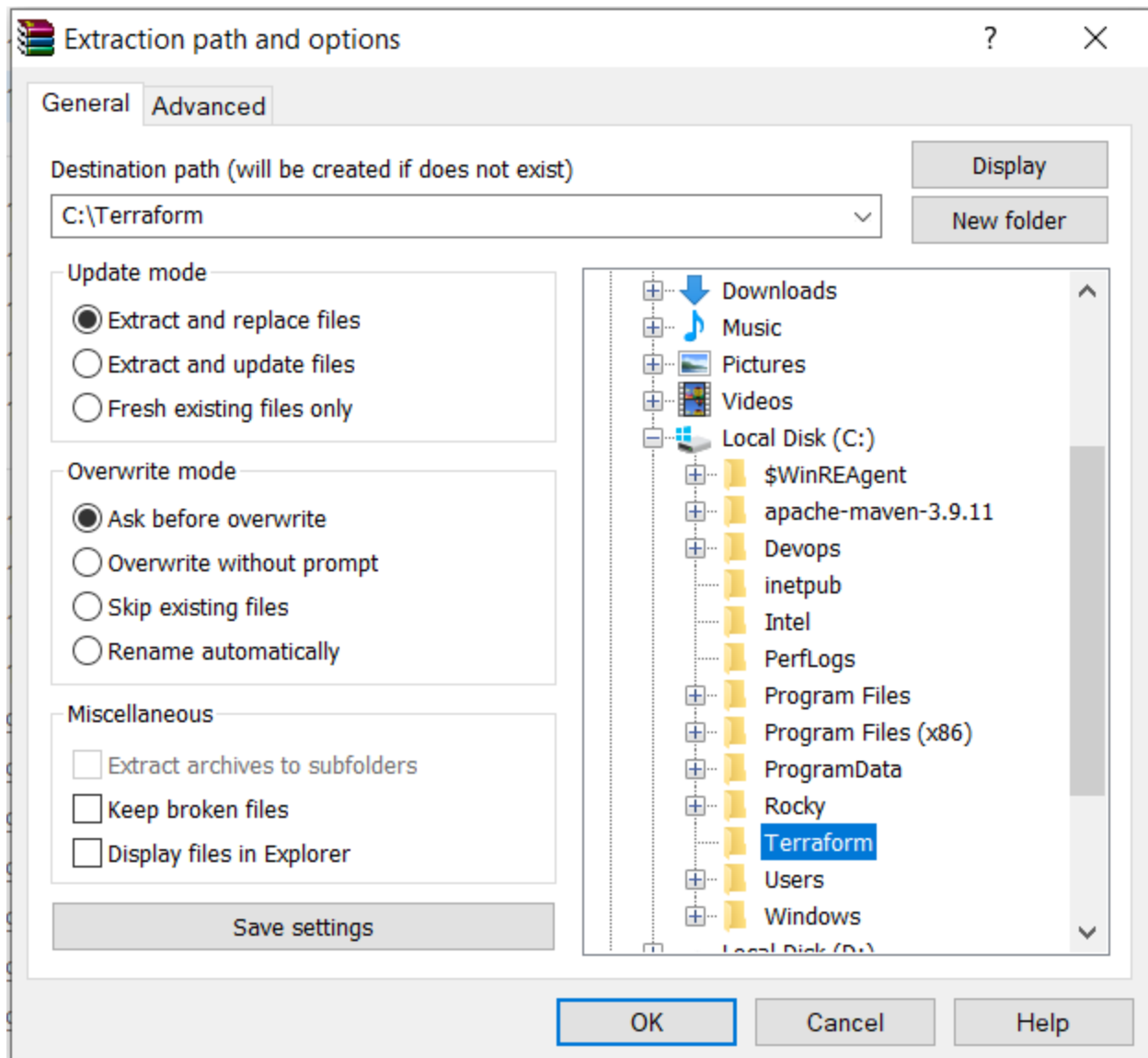
Windows

Binary download

Architecture	Version	Download
386	Version: 1.13.3	Download
AMD64	Version: 1.13.3	Download

Then download and zip file is downloaded

Then extract the zip file in windows in C drive and then create a folder with name is terraform



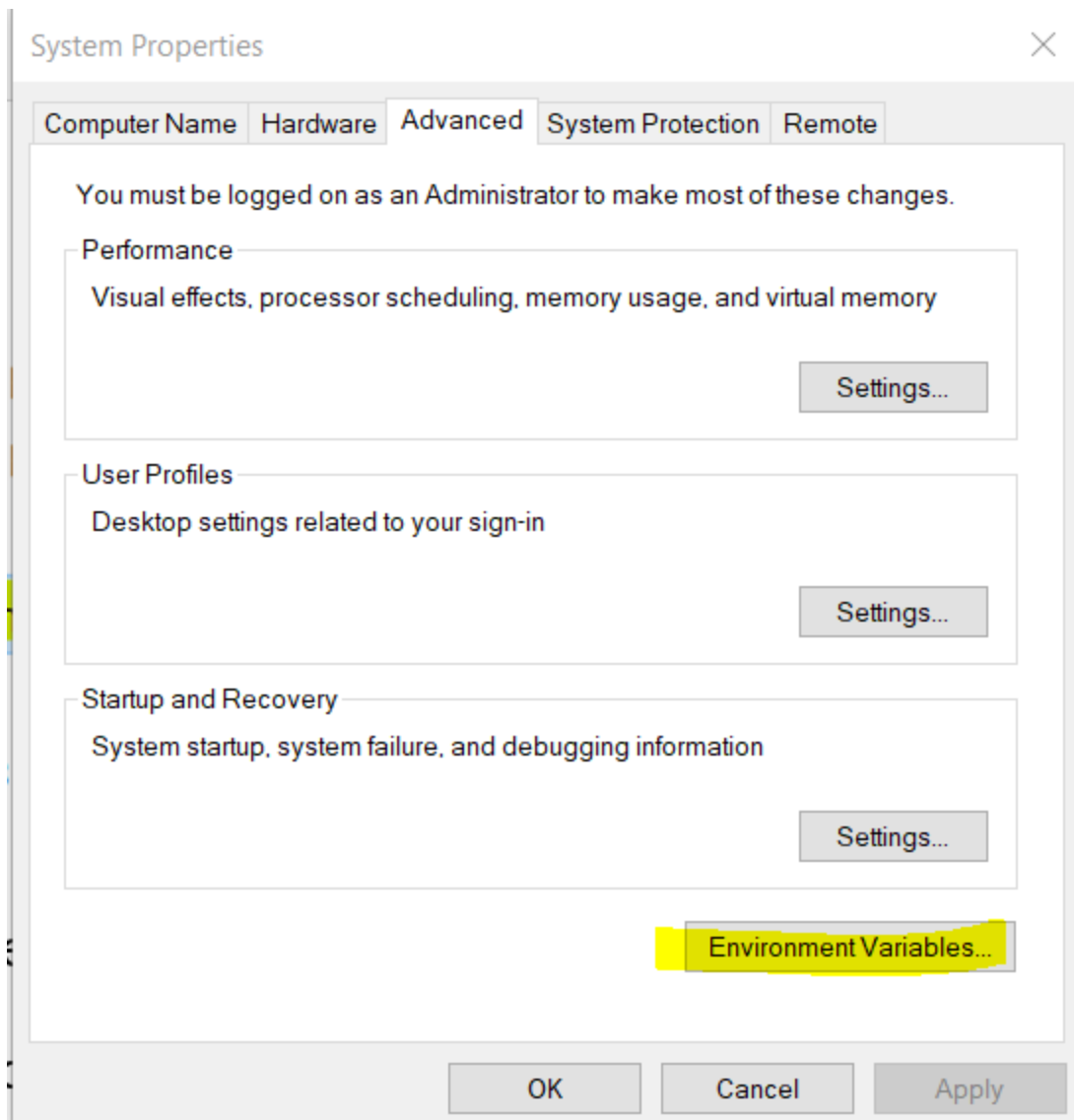
Then click ok and go to C drive inside terraform folder

PC > Local Disk (C:)

Name	Date modified	Type	Size
apache-maven-3.9.11	7/12/2025 6:30 PM	File folder	
Devops	9/18/2025 1:38 PM	File folder	
inetpub	7/6/2025 3:58 PM	File folder	
Intel	9/16/2025 11:50 PM	File folder	
PerfLogs	12/7/2019 2:44 PM	File folder	
Program Files	9/16/2025 8:54 AM	File folder	
Program Files (x86)	9/16/2025 9:32 AM	File folder	
Rocky	9/4/2025 9:10 PM	File folder	
Terraform	9/18/2025 1:56 PM	File folder	
Users	7/6/2025 1:42 PM	File folder	
Windows	9/16/2025 11:50 PM	File folder	

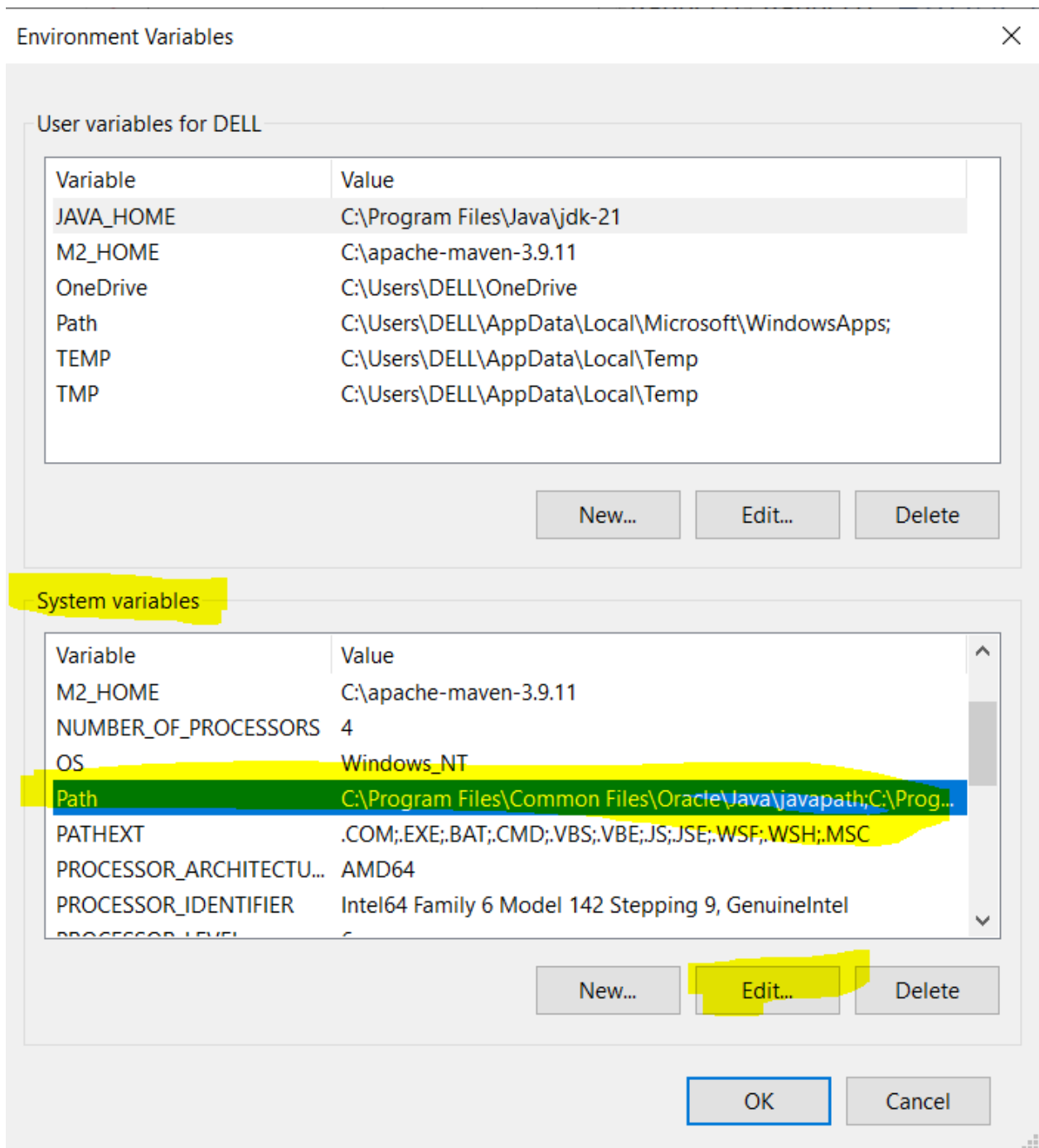
Go to inside terraform folder then copy the path

Once the copy the path go windows search bar type environment variable
then click on environment variable



Then new windows is open

Select system variable and click on path option and make it edit



Then click on new & then the terraform location path and then ok

Edit environment variable



C:\Program Files\Common Files\Oracle\Java\javapath
C:\Program Files (x86)\VMware\VMware Workstation\bin\
%SystemRoot%\system32
%SystemRoot%
%SystemRoot%\System32\Wbem
%SYSTEMROOT%\System32\WindowsPowerShell\v1.0\
%SYSTEMROOT%\System32\OpenSSH\
C:\Program Files\Git\cmd
C:\Program Files\Java\jdk-21\bin
C:\apache-maven-3.9.11\bin

New

Edit

Browse...

Delete

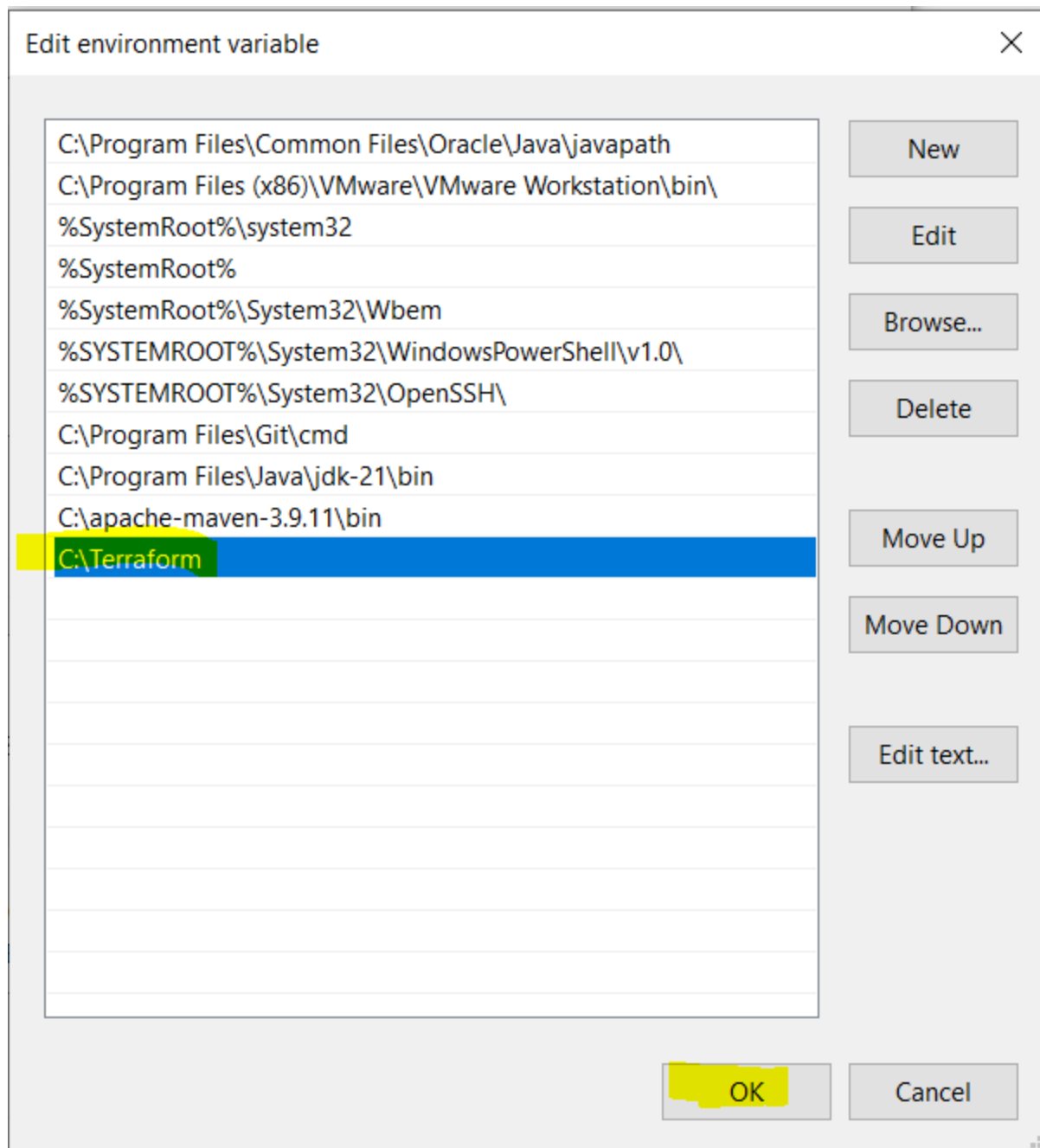
Move Up

Move Down

Edit text...

OK

Cancel



Now go to windows search bar type cmd
then command prompt is open

Run the command
terraform --version

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>terraform --version
Terraform v1.13.3
on windows_386

C:\Users\DELL>
C:\Users\DELL>
C:\Users\DELL>
```

If this output is not visible then restart the windows

Now Successfully installed Terraform on windows OS

Now Install Terraform on Amazon Linux

Login to AWS Console then Go to EC2 Service Launch 1
EC2 instance

Name:- Terraform_Server

AMI:- Amazon-Linux2023

Instance Type:- t3.micro

Key:- Terraform_Key

VPC:- Default

AZ:- 1a

SG:- Terraform_SG

Open port

SSH and HTTP

Storage:- 10GB

The screenshot displays the AWS Management Console interface. The top navigation bar shows the AWS logo, a search bar, and account information for 'Asia Pacific (Mumbai)' with Account ID 3091-4949-3821. The left sidebar lists various services, with 'EC2' selected. The main content area features the 'Amazon Elastic Compute Cloud (EC2)' dashboard, which includes a 'Launch a virtual server' button and a 'View dashboard' button. Below the dashboard, there are sections for 'Benefits and features' and 'Additional actions'. The bottom section of the screenshot shows the 'Instances (1)' list, which contains one instance named 'Terraform_Server' with Instance ID 'i-015fd47df9eaacca3', Instance state 'Running', Instance type 't3.micro', and Status check 'Initializing'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Avail
Terraform_Server	i-015fd47df9eaacca3	Running	t3.micro	Initializing	View alarms +	ap-sc

Now to instance via putty

Login User:- ec2-user

#sudo su -

```
Authenticating with public key "Imported-Openssh-Key"

• MobaXterm Personal Edition v26.4 •
(SSH client, X server and network tools)

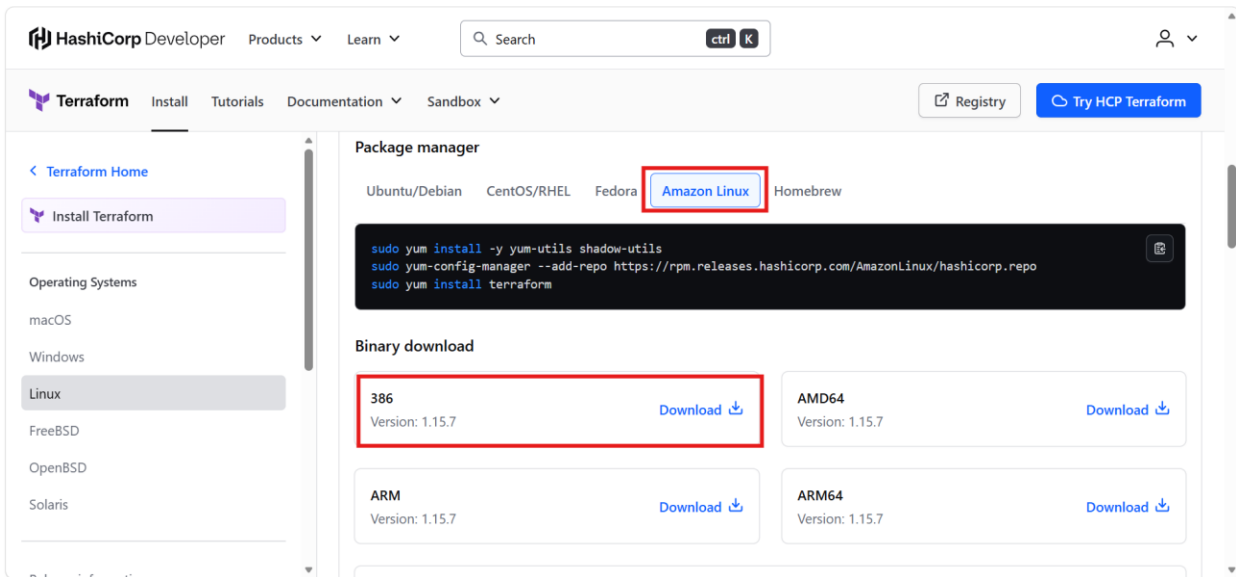
➤ SSH session to ec2-user@3.108.219.42
• Direct SSH : ✓
• SSH compression : ✗
• SSH-browser : ✓
• X11-forwarding : ✗ (disabled or not supported by server)
➤ For more info, ctrl+click on help or visit our website.

#
##### Amazon Linux 2023
#####
\###|
\##/
V~' -> https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-3-255 ~]$
[ec2-user@ip-172-31-3-255 ~]$
[ec2-user@ip-172-31-3-255 ~]$ sudo su -
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
[root@ip-172-31-3-255 ~]#
```

REGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

Now install Terraform on Amazon-Linux
go to browser type terraform.io



Now set the hostname

```
#hostnamectl set-hostname terraformsrv.example.com
```

```
#hostname
```

Now, Copy the binary

```
#wget
```

```
https://releases.hashicorp.com/terraform/1.15.7/terraform_1.15.7_linux_386.zip
```

```
#ls
```

```
[root@ip-172-31-3-255 ~]#  
[root@ip-172-31-3-255 ~]# hostnamectl set-hostname terraformsrv.example.com  
[root@ip-172-31-3-255 ~]#  
[root@ip-172-31-3-255 ~]# bash  
[root@terraformsrv ~]#  
[root@terraformsrv ~]#  
[root@terraformsrv ~]# wget https://releases.hashicorp.com/terraform/1.15.7/terraform_1.15.7_linux_386.zip  
--2026-07-02 10:54:50-- https://releases.hashicorp.com/terraform/1.15.7/terraform_1.15.7_linux_386.zip  
Resolving releases.hashicorp.com (releases.hashicorp.com)... 18.172.64.46, 18.172.64.101, 18.172.64.38, ...  
Connecting to releases.hashicorp.com (releases.hashicorp.com)|18.172.64.46|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 32184628 (31M) [application/zip]  
Saving to: 'terraform_1.15.7_linux_386.zip'  
  
terraform_1.15.7_linux_386.zip      100%[=====] 30.69M  --.-KB/s  
2026-07-02 10:54:50 (474 MB/s) - 'terraform_1.15.7_linux_386.zip' saved [32184628/32184628]  
  
[root@terraformsrv ~]# ls  
terraform_1.15.7_linux_386.zip  
[root@terraformsrv ~]#  
[root@terraformsrv ~]#
```

```
# unzip terraform_1.15.7_linux_386.zip
```

```
[root@terraformsrv ~]#  
[root@terraformsrv ~]# unzip terraform_1.15.7_linux_386.zip  
Archive:  terraform_1.15.7_linux_386.zip  
  inflating: LICENSE.txt  
  inflating: terraform  
[root@terraformsrv ~]# ls  
LICENSE.txt  terraform  terraform_1.15.7_linux_386.zip  
[root@terraformsrv ~]#
```

Now crate one directory inside /root

```
# mkdir /root/terra_project
```

```
#ls
```

```
# cd terra_project
```

```
[root@terraformsrv ~]#  
[root@terraformsrv ~]# mkdir /root/terra_project  
[root@terraformsrv ~]#  
[root@terraformsrv ~]#  
[root@terraformsrv ~]# ls  
LICENSE.txt  terra_project  terraform  terraform_1.15.7_linux_386.zip  
[root@terraformsrv ~]#  
[root@terraformsrv ~]# cd terra_project/  
[root@terraformsrv terra_project]#  
[root@terraformsrv terra_project]# pwd  
/root/terra_project  
[root@terraformsrv terra_project]#
```

```
#cd  
#rm -rf terraform_1.15.7_linux_386.zip  
#ls
```

```
[root@terraformsrv ~]# rm -rf terraform_1.15.7_linux_386.zip  
[root@terraformsrv ~]#  
[root@terraformsrv ~]# ls  
LICENSE.txt  terra_project  terraform  
[root@terraformsrv ~]#
```

Now move the terraform directory to

```
# mv terraform /usr/local/bin  
#ls  
# terraform --version
```

```
[root@terraformsrv ~]# mv terraform /usr/local/bin  
[root@terraformsrv ~]# ls  
LICENSE.txt  terra_project  
[root@terraformsrv ~]#  
[root@terraformsrv ~]# terraform --version  
Terraform v1.15.7  
on linux_386  
[root@terraformsrv ~]#
```

Note: - In amazon linux 2023 aws cli package is by default installed

```
#aws --version
```

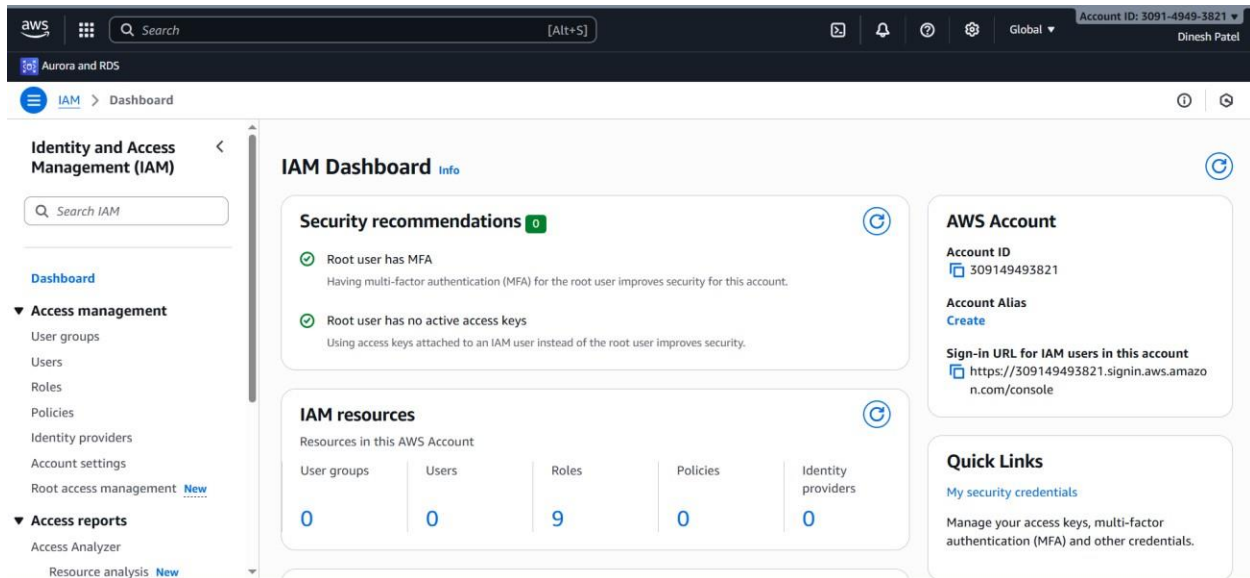
```

[root@terraformsrv ~]#
[root@terraformsrv ~]# aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.35-68.129.amzn2023.x86_64 source/x86_64.amzn.2023
[root@terraformsrv ~]#
[root@terraformsrv ~]# cat /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:amzn"
PRETTY_NAME="Amazon Linux 2023.12.20260629"
ANSI_COLOR="0;33"
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"
[root@terraformsrv ~]#

```

Now to run terraform script to automate to aws infra we need one IAM user and roles

Go to aws console search IAM Service create one IAM user



User Name:- terraform_user

IAM > Users > Create user

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

User details

User name

terraform_user

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☒ Provide user access to the AWS Management Console - optional

If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ Specify a user in Identity Center - Recommended

We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ I want to create an IAM user

We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

Console password

☐ Autogenerated password

You can view the password after you create the user.

☒ Custom password

Enter a custom password for the user.

☐ Show password

Users (1) Info

Refresh

Delete

Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

< 1 > ⚙

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in
☐	terraform_user	/	0	-	-	-	-

Now attached permission of this user is AdministratorAccess

Permissions policies (1/1523)

Search

Filter by Type

All types

< 1 2 3 4 5 6 7 ... 77 > ⚙

☐	Policy name	Type	Attached entities
☐	AccessAnalyzerServiceRolePolicy	AWS managed	0
☐	AccountAccessManagerServiceRolePolicy	AWS managed	0
☐	AccountManagementFromVercel	AWS managed	0
☒	AdministratorAccess	AWS managed - job function	0
☐	AdministratorAccess-Amplify	AWS managed	0
☐	AdministratorAccess-AWSElasticBeanstalk	AWS managed	0
☐	AIDevOpsAgentAccessPolicy	AWS managed	0
☐	AIDevOpsAgentActionsPolicy	AWS managed	0
☐	AIDevOpsAgentFullAccess	AWS managed	0

Add Permissions

Now, add terraform_user security credentials

To add access key & secret access key

Now go to terraform_server and run one command

aws configure --profile terraform_user

```
[root@terraformsrv ~]#  
[root@terraformsrv ~]# aws configure --profile terraform_user  
AWS Access Key ID [None]:
```

Add AWS Access Key & Secret Access key

Terraform setup is now complete. You can use it to manage infrastructure as code (IaC) and automate cloud resource provisioning efficient.

*******Thank You*******